Part 1 Introduction

This lab focuses on designing and extending a five-stage pipelined RISC-V processor, divided into four parts: (1) Enhancing the Control Unit, (2) Implementing the remaining RISC-V M instructions (mulh, mulhu, mulhsu), (3) Implementing Bypassing, and (4) Integrating a Pipelined Mul/Div Unit. The first part completes all rv32i instructions and partially implements the rv32m extension to establish a fundamental understanding of the RISC-V datapath and control flow. The second part continues from Lab 1 by integrating the Iterative Mul/Div Unit into the processor to implement RISC-V M instructions. The third part focuses on the hazard problems discussed in class and designs a bypassing mechanism to reduce stalls and improve performance. The final part challenges the traditional five-stage pipeline by designing a seven-stage pipelined Mul/Div Unit. Finally, functionality verification and performance evaluation are conducted using ubmark to gain deeper insight into the sources and limitations of performance improvement in pipelined architectures.

Part 2 Design

(1) Enhancing the Control Unit && (2) Implementing the remaining RISC-V M instructions

Describe both implemented and why:

In the original implementation, only the bne and blt branch instructions were supported, and the signal any_br_taken_Xhl was composed from a limited set of conditions.To improve RISC-V ISA completeness, additional branch types were implemented. Specifically, the original any_br_taken_Xhl was replaced with explicit condition signals, includingbeq_taken_Xhl, bne_taken_Xhl, blt_taken_Xhl, bge_taken_Xhl = ~lt, bltu_taken_Xhl, and bgeu_taken_Xhl = ~ltu.This modification makes the branch logic more explicit and easier to debug, as each condition can be independently observed during simulation.Furthermore, the use of ge = ~lt and geu = ~ltu is both logically correct and hardware-efficient—it reuses existing comparator outputs and avoids introducing additional comparison units.

For the M-extension, the md_ control signals were expanded from a simplified version to a full set:md_mul, md_mulh, md_mulhsu, md_mulhu, md_div, md_divu, md_rem, and md_remu.Undefined states were explicitly assigned as 3'bxxx to enhance simulation observability.This enhancement enables complete support for the RISC-V M extension while maintaining a unified data path structure—results can be consistently written back through “high/low-half selection” or via a fixed-latency pipelined multiplier/divider unit, eliminating the need for additional post-processing logic.The new control scheme also integrates naturally with the existing mdm_l and mdm_u (low/high-half) selection signals.

Other standard instructions were implemented by analyzing the original data path design and referring to the riscv-isa.txt specification to ensure functional correctness and design completeness.

(3) Implementing Bypassing

Describe both implemented and why:

In riscvbyp-CoreDpath, compared with riscvstall-CoreDpath, forwarding paths were added to resolve data dependency issues.ex:EX/MEM → ID/EX**:** allows ALU computation results to be directly forwarded to the next instruction.MEM/WB → ID/EX: enables load results to be provided to subsequent instructions even before the write-back stage.The introduction of these forwarding paths allows instructions to execute continuously without unnecessary stalls, significantly improving pipeline throughput.

Additionally, the system compares the destination register (rd) with the source registers (rs1 and rs2) to determine whether a data dependency exists. If so, the new multiplexers op0_byp_mux and op1_byp_mux dynamically select the correct data source.This design ensures that the ALU stage can obtain the most up-to-date operand values without waiting for the write-back (WB) stage, thereby maintaining data correctness while improving execution efficiency.

In riscvbyp-CoreCtrl, a hazard detection and bypass control mechanism was introduced.This logic differentiates between cases where forwarding can safely occur and cases where a stall is still required.For example, ALU operation results can be forwarded safely, whereas load-use hazards—dependencies on data not yet retrieved from memory—still require a bubble insertion.The control unit generates the corresponding bypass control signals (such as byp_sel_Xhl and byp_sel_Mhl) to drive the datapath multiplexers, ensuring that each stage dynamically selects the correct data source and that the pipeline operates synchronously and without conflicts.Such selective forwarding allows the processor to minimize unnecessary stalls while maintaining correct execution behavior.

Furthermore, to support the forwarding logic, certain pipeline registers were extended to include control flags and tag information, such as the destination register index (rd) and the write-back enable flag (wb_en).These signals propagate along with the instruction through the pipeline to ensure that computed results are correctly identified and forwarded when needed.This design maintains alignment between data and control signals over time, preventing mis-forwarding or write-back conflicts.

(4) Integrating a Pipelined Mul/Div Unit

Describe both implemented and why:

In the riscvlong architecture, the five-stage pipeline framework and complete bypass mechanism of riscvbyp were preserved, but a pipelined multiplication and division unit (Pipelined Mul/Div Unit) was further integrated. This enhancement enables the processor to support the RISC-V M-extension and to handle multi-stage multiply/divide operations without stalling the overall pipeline.

In implementation, the original single-cycle ALU of riscvbyp was extended by adding a new multiply/divide module with fixed multi-stage latency.Within the datapath (riscvlong-CoreDpath.v), the riscvlong-CoreDpathPipeMulDiv.v module implements a pipelined Mul/Div unit with multiple register stages, allowing new multiply/divide instructions to be issued even before previous ones complete, thus avoiding full-pipeline stalls. Additionally, an extended stall/bubble control pipeline was introduced to ensure that when the Mul/Div result is not yet available, the pipeline can correctly block or forward data to maintain consistency.

Because the Mul/Div unit has a fixed delay (typically 3–5 stages), the overall pipeline depth is effectively extended into a **l**ong pipeline structure of about seven stages.In the CoreDpath module, multiple MD registers (e.g., MD1, MD2, MD3) were added to carry destination register tags (rd) and control signals, ensuring that multi-stage results are correctly matched with their originating instructions.To further enhance efficiency, a tail-end bypass was implemented, allowing the final Mul/Div result to be forwarded to subsequent instructions before the write-back (WB) stage.

In the CoreCtrl module, new control fields and encodings were added for the M-extension:md_fn distinguishes between operations such as mul, mulh, mulhsu, mulhu, div, divu, rem, and remu.mdm_sel selects whether to output the high half (HI) or low half (LO) of the result.The hazard and bubble control logic were also updated, enabling the pipeline to automatically determine whether a Mul/Div result can be used immediately or must wait for completion.

Part 3 Testing Methodology

Add a corner.S file to the project containing the bypass/corner-case tests and build it. First run the tests on riscvstall and record the instruction count and cycle count for each case; then run the same tests with riscvbyp using identical inputs and compare the IPC between the two to determine whether bypassing significantly reduces stalls and improves execution efficiency.

Corner case:

```
// S1) load-use
la    x1, tdata_0
lw    x5, 0(x1)
add   x6, x5, x0
TEST_CHECK_EQ( x6, 0x000000ff )
// S2) RAW
addi  x7,  x0, 1
addi  x7,  x7, 2
add   x8,  x7, x0
TEST_CHECK_EQ( x8, 3 )
// S3) Negative sgift base
la    x9,  tdata_0
addi  x9,  x9, -32
lw    x10, 32(x9)
TEST_CHECK_EQ( x10, 0x000000ff )
// S4) EX->Store-Data
la    x9, storebuf
addi  x5, x0, 0x7f
addi  x5, x5, 1
sw    x5, 0(x9)
lw    x10, 0(x9)
TEST_CHECK_EQ( x10, 0x00000080 )
```

```
// S5) EX->EX
addi  x3, x0, 1
addi  x4, x0, 2
add   x2, x3, x4
add   x6, x2, x0
TEST_CHECK_EQ( x6, 3 )
// S6) Last-writer wins
addi x1, x0, 5
nop
addi x1, x0, 7
add  x2, x1, x0
TEST_CHECK_EQ(x2, 7)
//S7)Store -> Load forwarding
la    x1, storebuf
li    x2, 0x1234
sw    x2, 0(x1)
lw    x3, 0(x1)
TEST_CHECK_EQ(x3, 0x00001234)
TEST_RISCV_END
```

```
qiu@LAPTOP-BVJ16IMQ:~/lab2/build$ ./riscvstall-sim +exe=../tests/build/vmh/riscvstall-corner.vmh +stats=1
WARNING: ../riscvstall/riscvstall-sim.v:154: $readmemh: The behaviour for reg[...] mem[N:0]; $readmemh("...", mem); chan
ged in the 1364-2005 standard. To avoid ambiguity, use mem[0:N] or explicit range parameters $readmemh("...", mem, start
, stop);. Defaulting to 1364-2005 behavior.
*** PASSED ***
-------------------------------------------
 STATS
-------------------------------------------
 status     =          1
 num_cycles =        137
 num_inst   =         57
 ipc        = 0.416058
../riscvstall/riscvstall-sim.v:335: $finish called at 1425 (1s)
qiu@LAPTOP-BVJ16IMQ:~/lab2/build$ ./riscvbyp-sim +exe=../tests/build/vmh/riscvstall-corner.vmh +stats=1
WARNING: ../riscvbyp/riscvbyp-sim.v:154: $readmemh: The behaviour for reg[...] mem[N:0]; $readmemh("...", mem); changed
in the 1364-2005 standard. To avoid ambiguity, use mem[0:N] or explicit range parameters $readmemh("...", mem, start, st
op);. Defaulting to 1364-2005 behavior.
*** PASSED ***
-------------------------------------------
 STATS
-------------------------------------------
 status     =          1
 num_cycles =         61
 num_inst   =         57
 ipc        = 0.934426
../riscvbyp/riscvbyp-sim.v:335: $finish called at 665 (1s)
```

Both simulators pass the tests with the same inputs (*** PASSED ***), but riscvbyp clearly uses far fewer cycles than riscvstall — *riscvstall* took 137 cycles for 57 instructions (IPC = 0.416), while *riscvbyp* took only 61 cycles for 57 instructions (IPC = 0.934), saving about 76 cycles (≈55%) and increasing IPC by more than twofold. This indicates your implemented bypass paths effectively reduced RAW (read-after-write) / data-propagation stalls for this set of corner cases, bringing the average cycles per instruction close to 1.

PART4 Evaluation

bench	core	num_cycle	num_inst	ipc
vvadd	stall	1778	1064	0.598425
vvadd	byp	1447	1064	0.735314
cmplx-mult	stall	18687	2949	0.157810
cmplx-mult	byp	16685	2949	0.176746
Masked-filter	stall	21032	6964	0.331114
Masked-filter	byp	17701	6964	0.393424
Bin-search	stall	3258	1126	0.345611
Bin-search	byp	1539	1126	0.731644

PART5 Discussion

From the results, bypassing consistently improves performance across all four benchmarks: cycle counts decrease and IPC increases in every case. The largest improvement is for Bin-search, where cycles drop by about 53% and IPC increases by roughly 2.12×. By contrast, the smallest improvement is for cmplx-mult, where cycles are reduced by only about 10.7% and IPC increases modestly (≈1.12×). This suggests that cmplx-mult is likely bounded by long-latency functional units or structural bottlenecks (e.g., multi-cycle multipliers or memory contention), in which case simple register forwarding yields limited benefit.

According to the Iron Law: Execution time=Instruction count *CPI*Cycle times the tradeoffs between the two strategies can be summarized as follows. Stalling (a simpler design) has the advantage of simpler control and datapath logic and typically smaller impact on cycle time; its drawback is increased CPI because idle cycles are inserted to satisfy dependencies, and if the CPI increase outweighs any cycle-time advantage, total execution time will rise. Bypassing (forwarding) reduces CPI by eliminating dependency-induced stalls and therefore provides large gains for in-order pipelines with frequent short dependencies; its cost is added forwarding multiplexers and selection logic, which can slightly increase cycle time and raise design/verification complexity, area, and power.

In short, bypassing is beneficial when the reduction in CPI outweighs any increase in cycle time caused by the extra hardware. In our measurements, the CPI reduction from bypassing far exceeds any likely cycle-time penalty, so overall execution time decreases significantly.

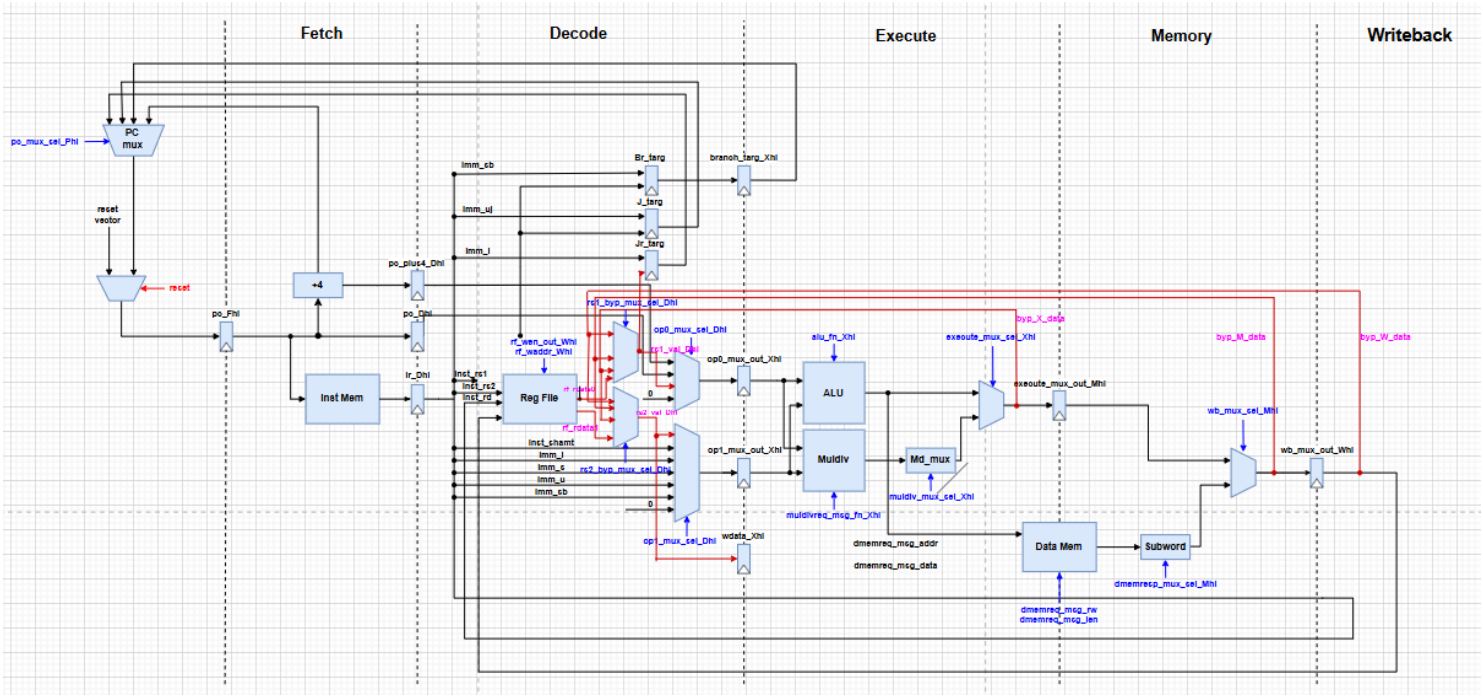

Figure. Bypassing datapath